

0.1. Диаграмма компонентов

0.1.1. Компонент

Компонент – модульная и замещаемая часть системы, инкапсулирующая ее содержимое.

Компонент как чёрный ящик, внешнее поведение которого полностью определяется его предоставляемыми и требуемыми интерфейсами. Поэтому один компонент может быть заменён другим, поддерживающим тот же протокол.

Компоненты могут иметь атрибуты и операции и участвовать в отношениях ассоциации и обобщения. Компоненты – это структурированные классификаторы. У них может быть внутренняя структура, включающая части и соединители.

На диаграмме компонентов могут быть показаны компоненты, зависимости между ними и то, как компонентам назначаются классификаторы. Компонент отображается в виде прямоугольника со стереотипом «component» (компонент) и/или пиктограммой компонента в верхнем правом углу. У компонентов могут быть предоставляемые и требуемые интерфейсы и порты.

Компонент может иметь внутреннюю структуру. Части можно показать вложенными внутри компонента или находящимися снаружи и соединёнными с ним отношением зависимости. Обе формы синтаксически эквивалентны, хотя первая нотация более наглядна.

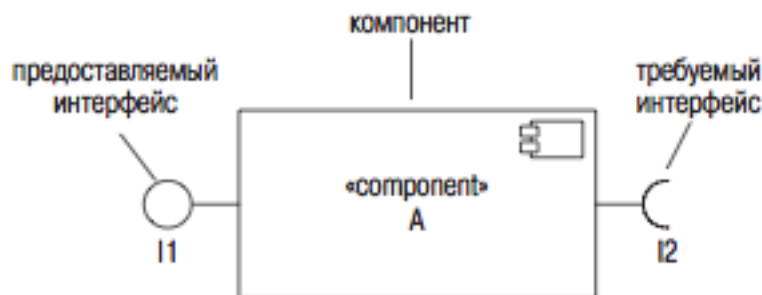


Рис. 4.1. Нотация компонента

Если у компонента есть внутренняя структура, как правило, он будет делегировать обязанности, определенные его интерфейсами, своим внутренним частям. На рис. 4.2 компонент А предоставляет интерфейс I1 и требует интерфейс I2. Он инкапсулирует две части типа b и c. Он делегирует поведение, описанное его предоставляемым и требуемым интерфейсами b и c соответственно.

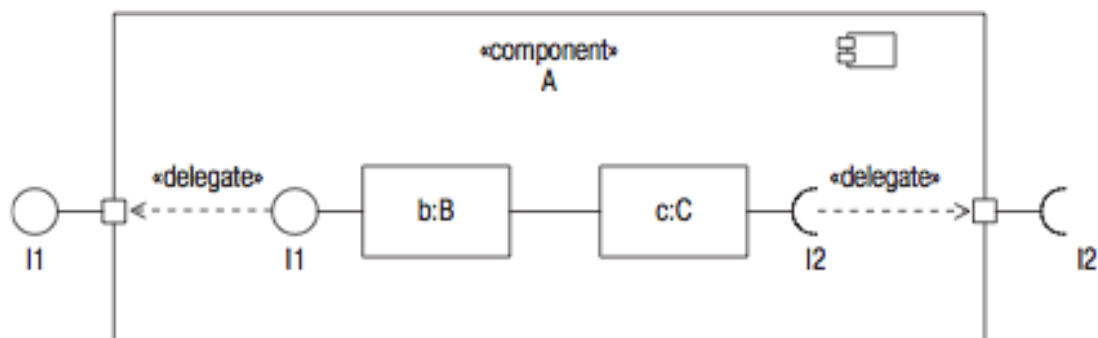


Рис. 4.2. Внутренняя структура компонента

0.1.2.Подсистемы

Подсистема – это компонент, действующий как единица декомпозиции большой системы. Подсистемы изображаются как компонент со стереотипом «*subsystem*».

Подсистема – это логическая конструкция, используемая для декомпозиции большой системы в управляемые части. Экземпляры самих подсистем *не могут* создаваться во время выполнения, но могут создаваться экземпляры их содержимого.

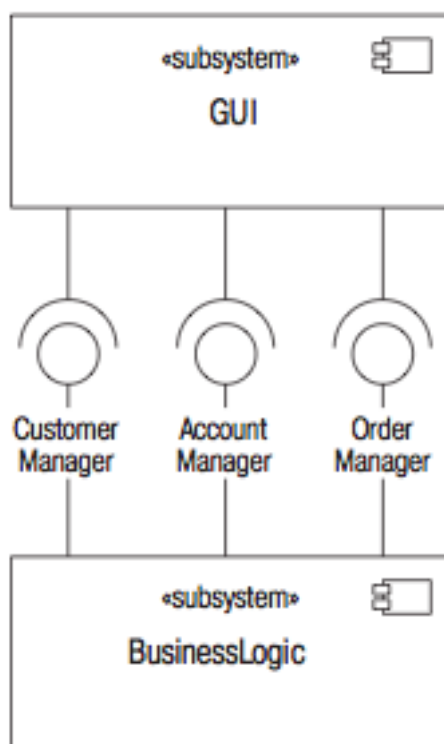


Рис. 4.3. Подсистемы и интерфейсы

0.2. Диаграмма пакетов

0.2.1. Пакеты

Пакет – это группирующая сущность. Это контейнер и владелец элементов модели. У каждого пакета есть своё пространство имён, в рамках которого все имена должны быть уникальными.

По сути, пакет – это универсальный механизм организации элементов модели (включая другие пакеты) и диаграмм в группы. Он может использоваться для следующих целей:

- предоставления инкапсулированного пространства имён, в рамках которого все имена должны быть уникальными;
- группировки семантически взаимосвязанных элементов;
- определения «семантической границы» модели;
- предоставления элементов для параллельной работы и управления конфигурацией. [SEP]

Пакеты позволяют создавать допускающую навигацию хорошо структурированную модель, обеспечивая возможность группировать сущности, имеющие близкие семантические связи. В модели можно устанавливать семантические границы, в пределах которых разные пакеты описывают разные аспекты функциональности системы.

Пакет определяет так называемое инкапсулированное пространство имён. Это означает, что пакет создаёт границу, в рамках которой имена всех элементов должны быть уникальными. Это также означает, что если элементу из одного пространства имён необходимо обратиться к элементу из другого пространства имён, он должен указать и имя необходимого элемента, и путь к этому элементу, чтобы его можно было найти в пространствах имён.

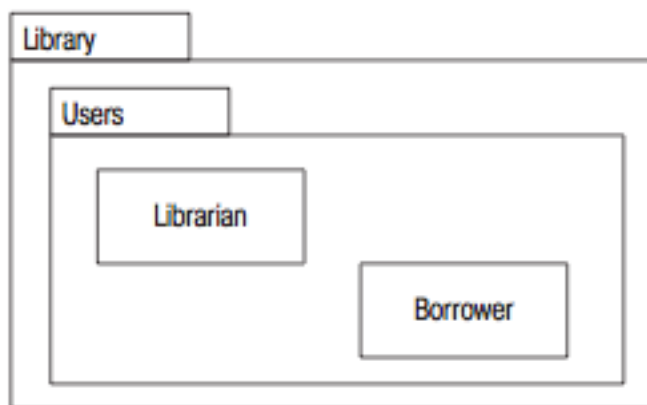


Рис. 4.4. Синтаксис вложения путём встраивания

0.2.2. Вложенные пакеты

Пакеты могут быть вложены в другие пакеты с любой глубиной вложенности. Однако обычно достаточно всего двух или трёх уровней. В противном случае модель может стать трудной для понимания и в ней будет сложно ориентироваться.

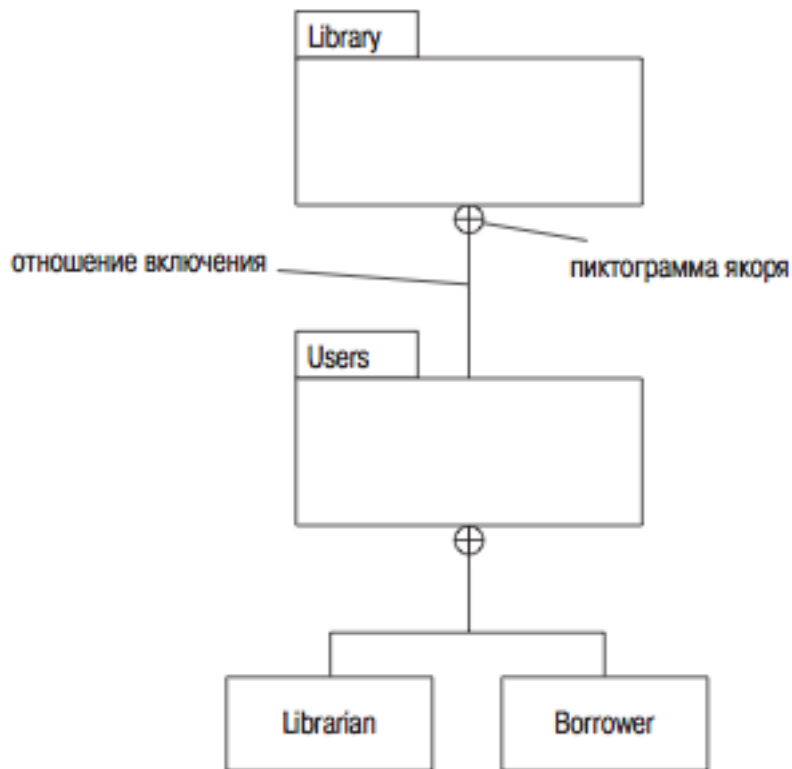


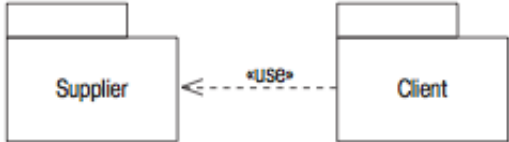
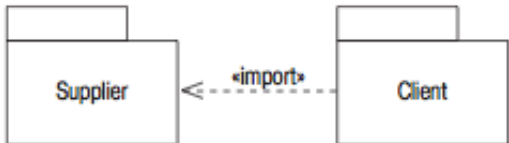
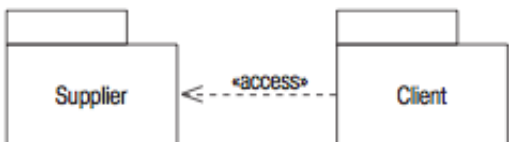
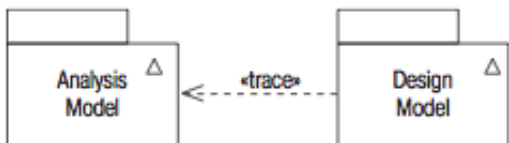
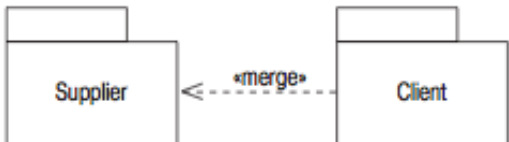
Рис. 4.5. Синтаксис сложного вложения

0.2.3.Зависимости пакетов

Между пакетами может быть установлено отношение зависимости.

И как всегда, при создании аналитической модели пакетов необходимо придерживаться простоты. Важнее получить правильный набор пакетов, чем широко применять такие возможности, как обобщение пакетов и стереотипы зависимостей. Все это можно добавить позже и только в том случае, если это сделает модель более понятной.

Отношения на диаграмме пакетов

Отношение зависимости пакетов	Семантика
	Элемент клиентского пакета некоторым образом использует открытый элемент пакета-поставщика – клиент зависит от поставщика. Если зависимость пакета показана без стереотипа, необходимо предполагать зависимость «use».
	Открытые элементы пространства имен поставщика добавляются как открытые элементы в пространство имен клиента. Элементы клиента могут организовывать доступ ко всем открытым элементам поставщика через неполные имена.
	Открытые элементы пространства имен поставщика добавляются как закрытые элементы в пространство имен клиента. Элементы клиента могут организовывать доступ ко всем открытым элементам поставщика с помощью неполных имен.
	«trace», как правило, представляет историческое развитие одного элемента в другую более развитую версию; обычно это отношение между моделями, а не элементами (межмодельное отношение).
	Открытые элементы пакета-поставщика объединяются с элементами клиентского пакета. Эта зависимость используется только при создании метамодели; в обычном ОО анализе и проектировании она не должна встречаться.

Отсутствие вложенных пакетов – один из гарантов простоты модели. Чем глубже что-то помещено в структуру вложенных пакетов, тем более непонятным оно становится.

0.3. Диаграмма развёртывания

Диаграмма развёртывания проецирует программную архитектуру на аппаратную архитектуру.

Диаграмма развёртывания определяет физическое оборудование, на котором будет выполняться программная система, а также описывает, как программное обеспечение развёртывается на это оборудование.

Диаграмма развёртывания проецирует программную архитектуру, созданную при проектировании, на исполняющую ее физическую архитектуру системы. В распределённых системах она моделирует распределение программного обеспечения по физическим узлам.

Существует две формы диаграмм развёртывания.

1. **Дескрипторная форма (descriptor form)** – содержит узлы, отношения между узлами и артефакты. Узел представляет тип оборудования (например, ПК). Аналогично артефакт представляет тип физического программного артефакта, например Java JAR-файл.
2. **Экземплярная форма (instance form)** – включает экземпляры узлов, отношения между экземплярами узлов и экземпляры артефактов. Экземпляры узлов представляют конкретную, идентифицируемую часть оборудования (например, ПК Джима). Экземпляр артефакта представляет конкретный экземпляр типа программного обеспечения, например, конкретный JAR файл. Если детали конкретных экземпляров неизвестны (или неважны), могут использоваться анонимные экземпляры.

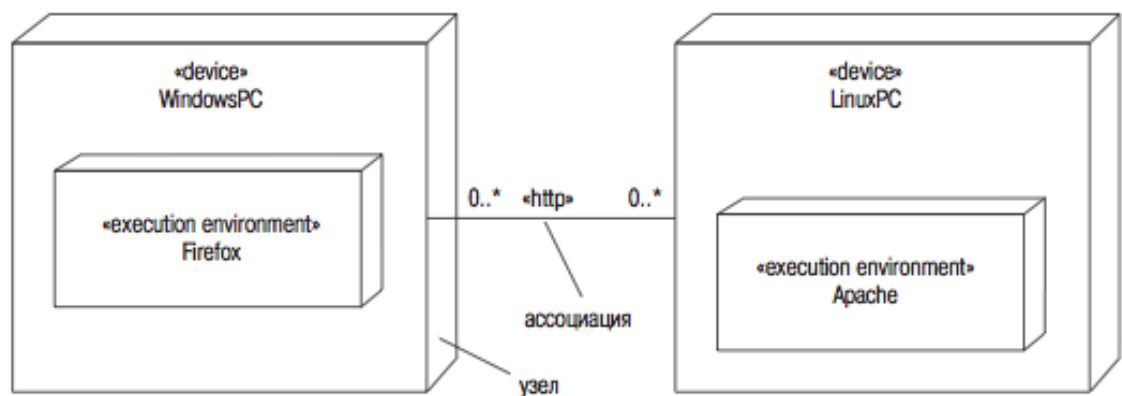


Рис. 4.6. Дескрипторная форма диаграммы развёртывания

Дескрипторная форма диаграмм развёртывания хороша для моделирования типа физической архитектуры, а экземплярная форма – для моделирования фактического развёртывания этой архитектуры на конкретном сайте.

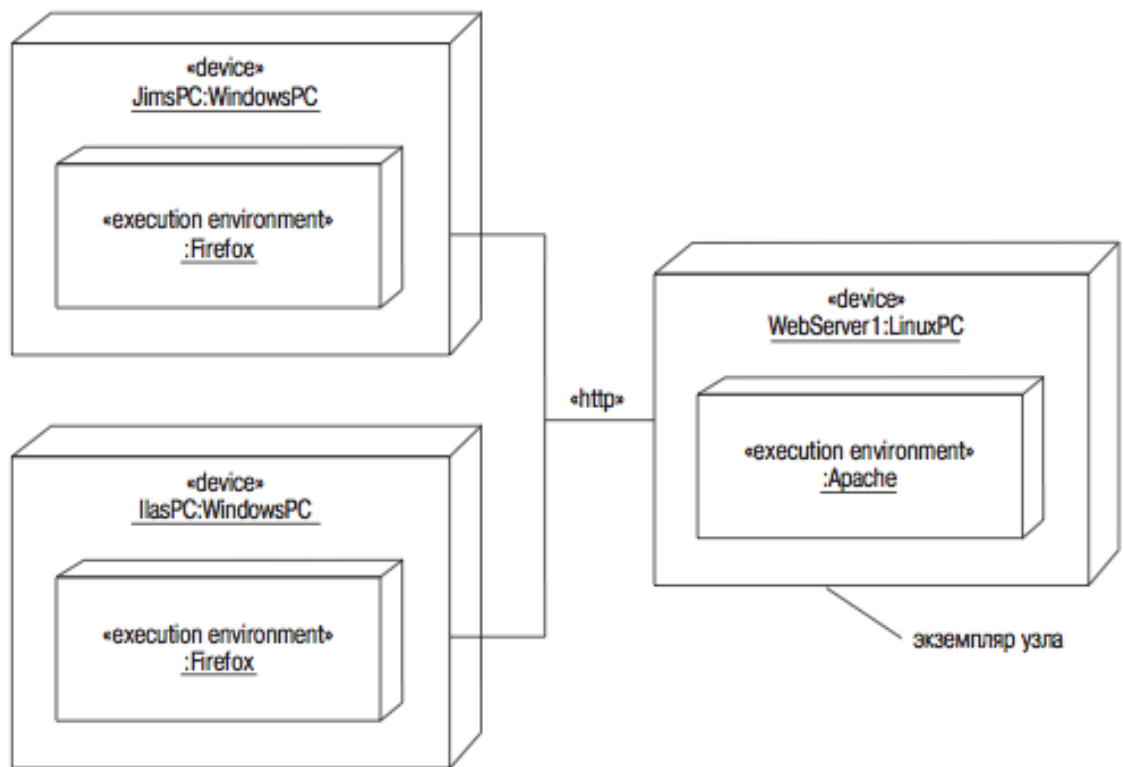


Рис. 4.7. Экземплярная форма диаграммы развёртывания

Для стереотипов можно придумать собственные пиктограммы, напоминающие реальное оборудование, и затем использовать эти символы на диаграмме развёртывания. Такой подход упрощает восприятие диаграммы. Пример полностью визуализированной дескрипторной формы диаграммы развёртывания приведён на рис. 4.8.

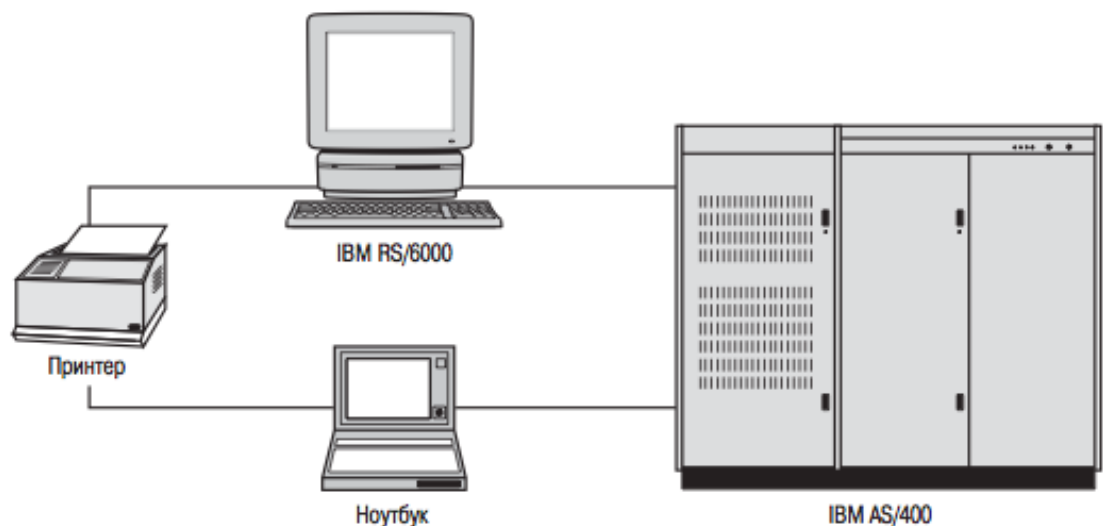


Рис. 4.8. Визуализированная дескрипторная форма диаграммы развёртывания

0.4.Задание на лабораторную работу.

- Изучить теоретическую часть лабораторной работы.

- Для выбранной предметной области определить структуру программной системы в виде диаграммы компонент.
- Оформить архитектуру программной системы в виде диаграммы пакетов.
- Сформировать техническую среду развёртывания программной системы в виде диаграммы развёртывания.
- Оформить отчёт по лабораторной работе

0.5. Диаграмма классов

0.5.1. Нотация классов в UML2

Визуальный синтаксис UML2 для класса очень богат. Чтобы синтаксис был управляемым, в UML2 существует понятие необязательных дополнений. Обязательной частью в визуальном синтаксисе является только ячейка с именем класса. Все остальные ячейки и дополнения необязательны. Для справки на рис. 5.1. показано все.

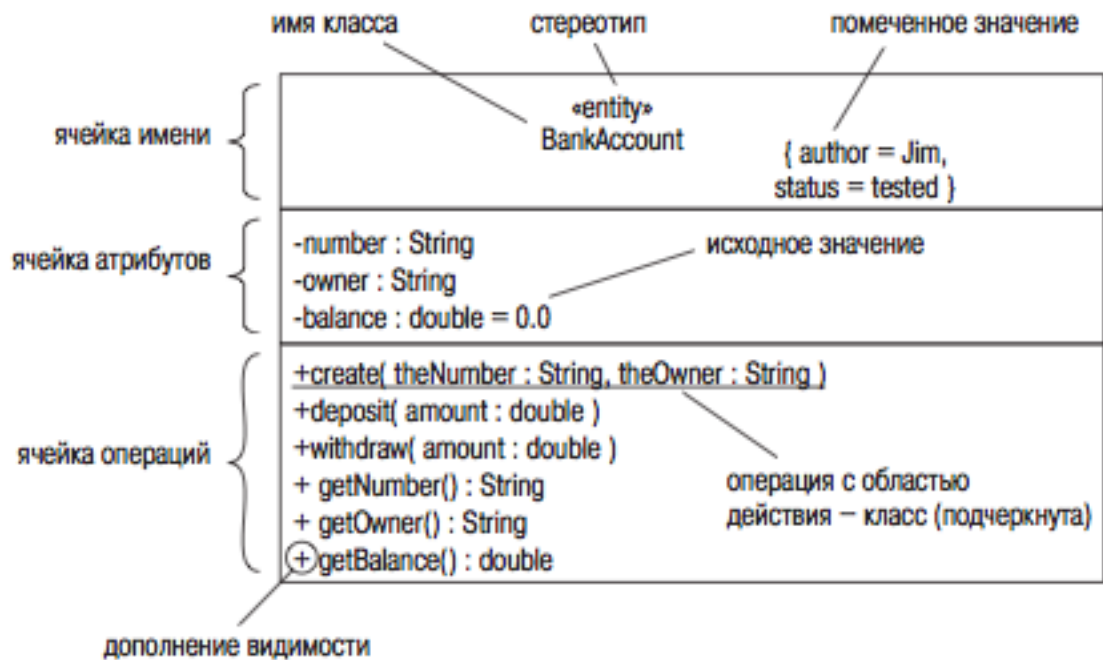


Рис. 5.1. Нотация классов в UML2

В аналитических моделях обычно необходимо показывать только:

- имя класса;
- ключевые атрибуты;
- ключевые операции;
- стереотипы (если они приносят пользу делу).

Обычно *не* показывают следующее:

- помеченные значения;
- параметры операций;
- видимость;
- исходные значения (если только они не значимы для дела).

Используя общую структуру описания классов в языке UML2 возможно построить модели не только классов, но и таблиц баз данных и интерфейсов.

При использовании ячейки имени (обязательной для любой диаграммы классов) и ячейки атрибутов, опуская ячейку операций, получаем описание таблиц базы данных в нотации диаграммы классов.

При использовании ячейки имени и ячейки операций, опуская ячейку атрибутов, получаем описание интерфейсов, или описание структуры диалогового взаимодействия пользователя с системой, в нотации диаграммы классов.

0.5.2.Отношения

Отношения – это семантические (значимые) связи между элементами модели. Отношения – это способ объединения сущностей в UML2.

Обычно одна связь соединяет только два объекта. Однако UML2 допускает соединение нескольких объектов одной связью. Такую связь называют n-арной. Она изображается в виде ромба, от которого отходят линии к каждому из объектов-участников.

Спецификация UML2 допускает три разных способа отображения возможности навигации. Самое распространённое обозначение:

- все кресты опускаются;
- двунаправленные ассоциации изображаются *без* стрелок;
- однонаправленные ассоциации изображаются с одной стрелкой.



Рис. 5.2. Однонаправленная связь между объектами

Ассоциации – это отношения между классами. Аналогично связям, соединяющим объекты, ассоциации соединяют классы. Самое главное: для того чтобы между двумя объектами была связь, между классами этих объектов *должна* существовать ассоциация. Потому что связь – это экземпляр ассоциации. Так же как объект – экземпляр класса.

Семантика базовой, неуточненной ассоциации чрезвычайно проста: ассоциация между классами указывает на то, что между объектами этих классов могут устанавливаться связи. Существуют другие, более конкретные формы ассоциаций (агрегация и композиция).

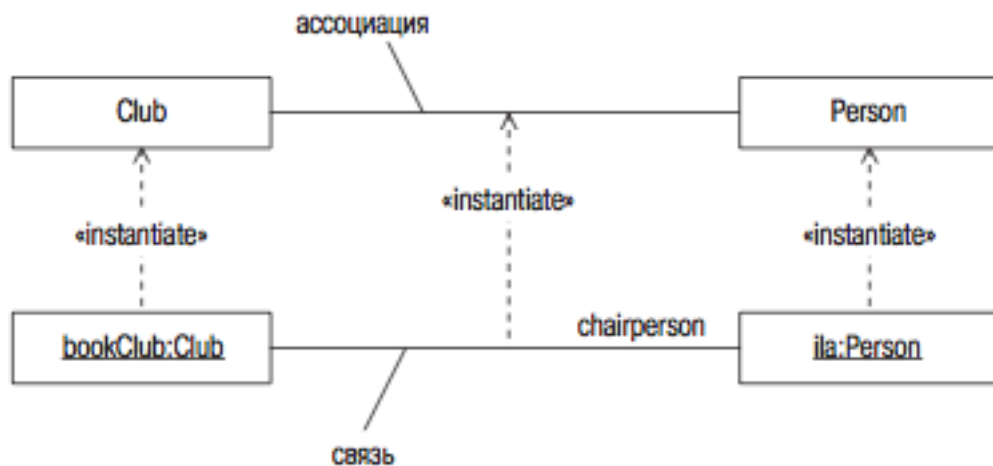


Рис. 5.3. Отношения между классами и объектами и между связями и ассоциациями

Ассоциации могут иметь:

- имя ассоциации
- имена ролей
- кратность
- возможность навигации

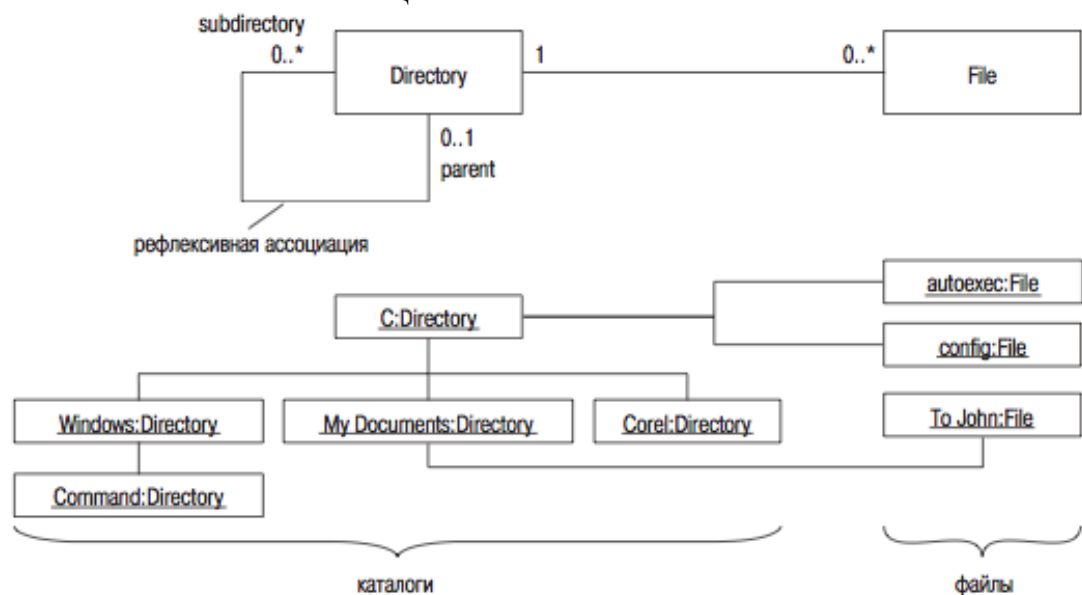


Рис. 5.4. Пример рефлексивной ассоциации: вверху – диаграмма классов, внизу – диаграмма объектов

0.5.3. Возможность навигации ^[L]_[SEP]

Возможность навигации (navigability) указывает на возможность прохода от объекта исходного класса к одному или более объектам в зависимости от кратности целевого класса. Смысл навигации в том, что

«сообщения могут посылаются только в направлении, в котором указывает стрелка».

Таблица 5.1
Стили представления возможности навигации в UML2

Стили представления возможности навигации в UML 2			
Синтаксис UML 2	Стиль 1: Строгое обозначение возможности навигации в UML 2	Стиль 2: Возможность навигации не обозначается	Стиль 3: Общепринятая практика
	Есть возможность навигации от А к В Есть возможность навигации от В к А		
	Есть возможность навигации от А к В Нет возможности навигации от В к А		
	Есть возможность навигации от А к В Возможность навигации от В к А не определена		Есть возможность навигации от А к В Нет возможности навигации от В к А
	Возможность навигации от А к В не определена Возможность навигации от В к А не определена	Возможность навигации от А к В не определена Возможность навигации от В к А не определена	Есть возможность навигации от А к В Есть возможность навигации от В к А
	Нет возможности навигации от А к В Нет возможности навигации от В к А		

0.5.4.Квалифицированные ассоциации

Квалифицированные ассоциации могут использоваться для превращения ассоциации n ко многим в ассоциацию n к одному путём задания одного объекта (или группы объектов) из целевого набора. Это очень полезные элементы модели, поскольку они показывают, как можно вести поиск или осуществлять навигацию к конкретным объектам коллекции.

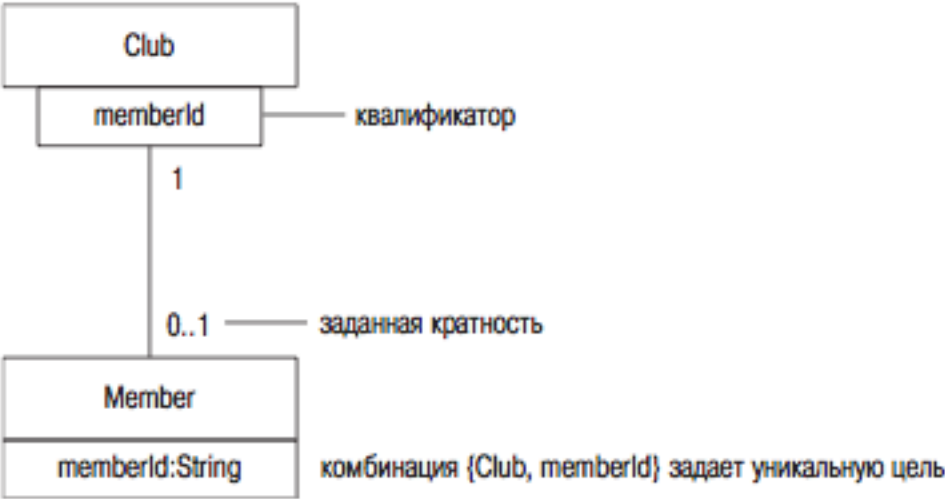


Рис. 5.5. Квалификатор превращает отношение один ко многим в отношение один к одному

0.5.5. Зависимость

Зависимость обозначает отношение между двумя или более элементами модели, при котором изменение одного элемента (поставщика) может повлиять или предоставить информацию, необходимую другому элементу (клиенту). Иначе говоря, клиент некоторым образом зависит от поставщика. Зависимости используются для моделирования отношений между классификаторами, когда один классификатор зависит от другого, но отношение не является ни ассоциацией, ни обобщением.

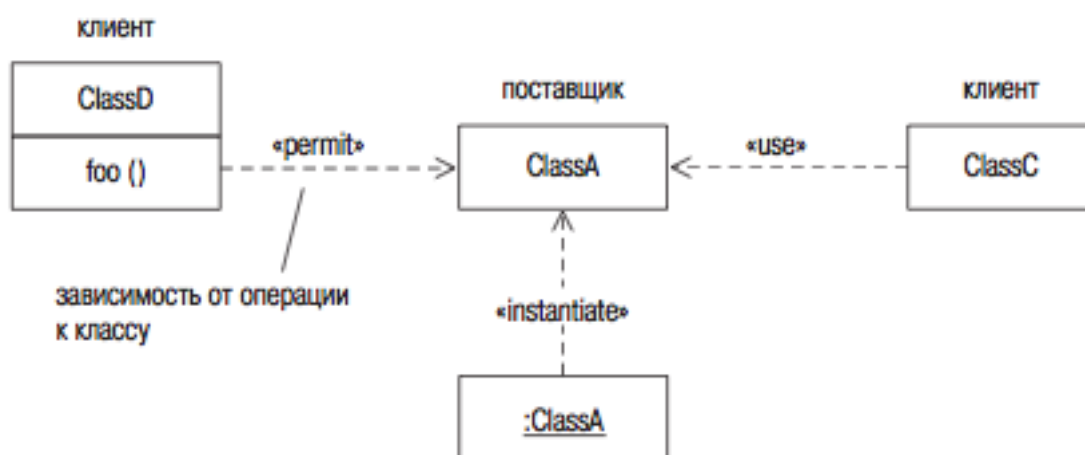


Рис. 5.6. Типы зависимостей

0.5.6. Обобщение

Обобщение – это отношение между более общей сущностью и более специальной сущностью.

Два элемента подчиняются принципу замещаемости: более специальный элемент может использоваться *везде*, где предполагается использование более общего элемента, без нарушения системы. Очевидно, что обобщение – намного более прочный тип отношений, чем ассоциация. В самом деле, обобщение подразумевает самый высокий уровень зависимости (и, следовательно, связанности) между двумя элементами.

0.5.7. Наследование классов

В иерархии обобщения кроется наследование между классами, посредством которого подклассы наследуют все возможности своих надклассов. Чтобы быть более специальными, подклассы наследуют:

- атрибуты;
- [SEP] операции;
- [SEP] отношения;
- [SEP] ограничения.

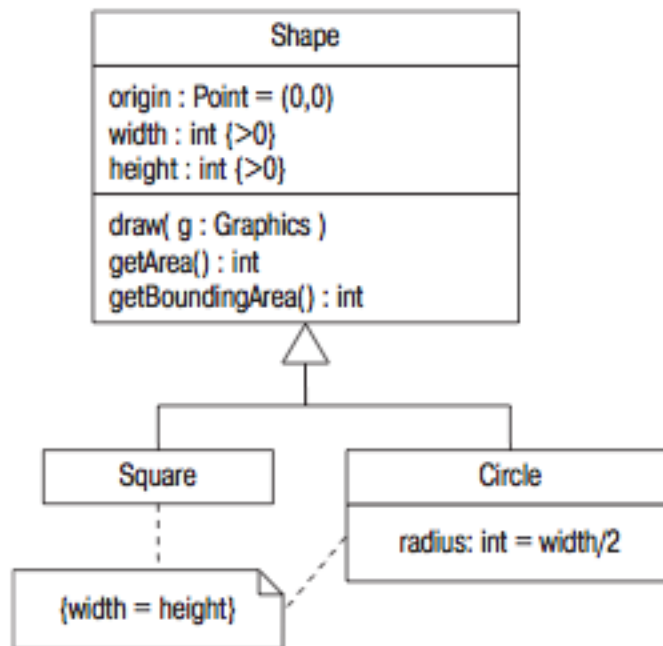


Рис. 5.7. Наследование характеристик суперкласса

0.5.8. Интерфейсы

Интерфейс определяет именованный набор открытых свойств.

Главная идея, лежащая в основе интерфейсов, – разделение *описания* функциональности (интерфейс) от ее *реализации* классификатором, таким как класс или подсистема. Создать экземпляр интерфейса невозможно. Он просто объявляет контракт, который может быть реализован классификаторами. Все, что реализует интерфейс, принимает и соглашается следовать определяемому интерфейсом контракту.

В интерфейсах также должны присутствовать описания семантики их возможностей (обычно в виде текста или псевдокода) как руководства для тех, кто будет их реализовывать.

Важно помнить, что интерфейс определяет только *описание* своих возможностей и что он *никогда* не включает в себе какой-либо конкретной реализации.



Рис. 5.8. Два варианта синтаксиса для предоставляемых интерфейсов

0.5.9.Порты

Порт группирует семантически связанный набор предоставляемых и требуемых интерфейсов. Он указывает на конкретную точку взаимодействия классификатора и его окружения.

Порты являются очень удобным способом структурирования предоставляемых и требуемых интерфейсов классификатора. Их также можно использовать для упрощения диаграммы. Например, на рис. 5,9 показан класс Viewer (средство просмотра), соединяющийся с портом presentation класса Book. Чтобы обеспечить возможность соединения портов, их предоставляемый и требуемый интерфейсы должны совпадать. Использование портов, очевидно, обеспечивает намного более краткое представление, чем отображение всех предоставляемых и требуемых интерфейсов, но может и усложнять чтение диаграмм.

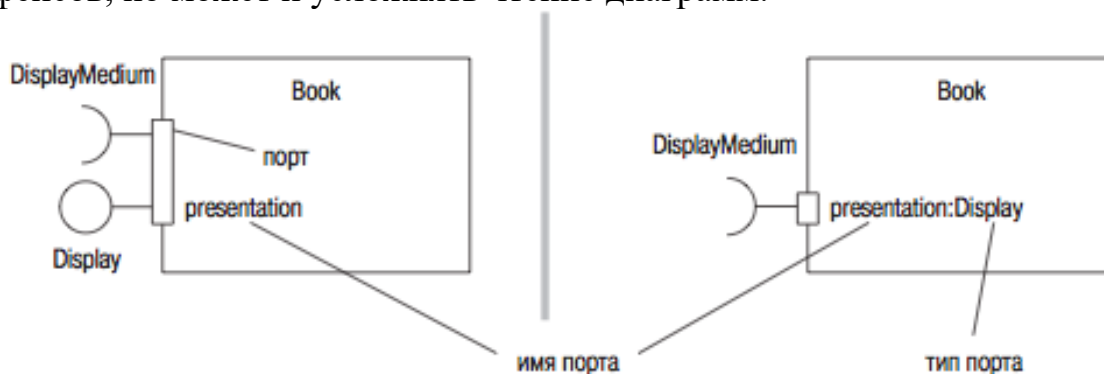


Рис. 5.9. Два варианта нотации порта

0.6.Задание на лабораторную работу.

- Изучить теоретическую часть лабораторной работы.

- Для произвольной программной системы сформировать статическую структуру классов, используя различные средства для представления взаимодействия классов.
- Для разработанной структуры классов сформировать структуру базы данных в нотации диаграммы классов.
- Для разработанной структуры классов сформировать пользовательский интерфейс в нотации диаграммы классов.
- Оформить отчёт по лабораторной работе

0.7. Диаграммы взаимодействий

Диаграммы взаимодействий UML2 могут использоваться для моделирования любого типа взаимодействия между экземплярами классификаторов. В частности, в реализации прецедентов диаграммы взаимодействий используются для моделирования взаимодействий между объектами, реализующими прецедент или его часть. Существует четыре разных типа диаграмм взаимодействий, каждый из которых делает акцент на различных аспектах взаимодействия.

Диаграммы последовательностей (sequence diagrams) акцентируют внимание на временной упорядоченности сообщений. Обычно пользователи лучше понимают диаграммы последовательностей, чем коммуникационные диаграммы, поскольку они намного легче читаются. Как правило, коммуникационные диаграммы очень быстро загромождаются.

Коммуникационные диаграммы (communication diagrams) выделяют структурные отношения между объектами и очень полезны при анализе, особенно для создания эскиза совместной работы объектов. В UML2 эти диаграммы предлагают только лишь подмножество функциональности диаграмм последовательностей.

Диаграммы обзора взаимодействий (interaction overview diagrams) показывают, как сложное взаимодействие реализуется рядом простых взаимодействий. Это особый случай диаграммы деятельности, в которой узлы ссылаются на другие взаимодействия. Они полезны для моделирования потока управления системы.

Временные диаграммы (timing diagrams) обращают внимание на фактическое время взаимодействия. Их основное назначение – по□ мочь оценить временные затраты.

0.8. Диаграммы последовательностей

Диаграммы последовательностей представляют взаимодействия между линиями жизни как упорядоченную последовательность событий. Это самая богатая и гибкая форма диаграммы взаимодействий.

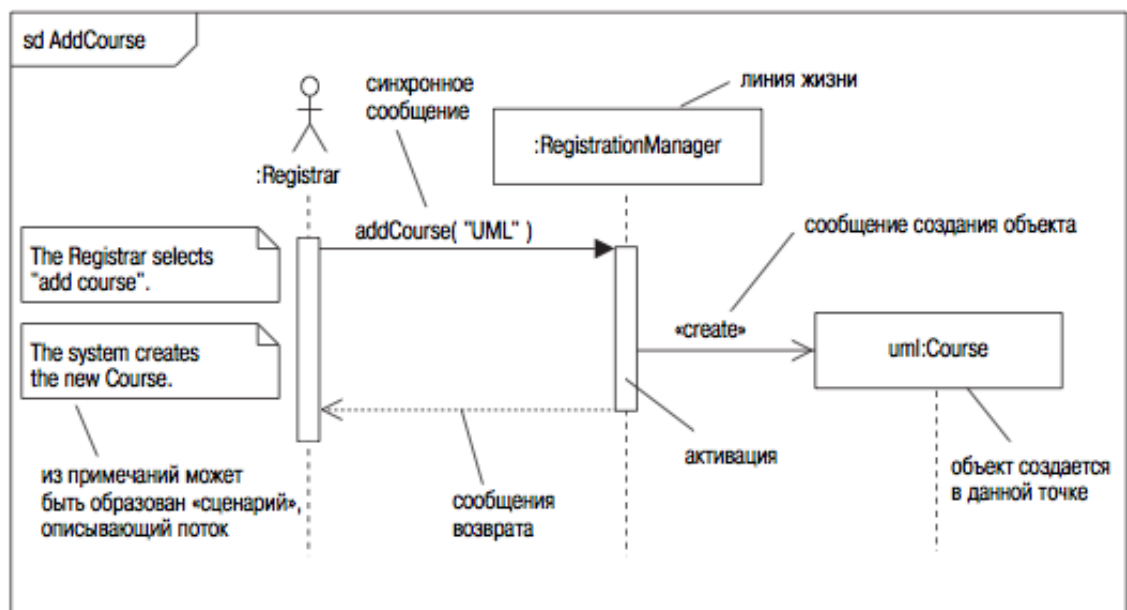


Рис. 6.1. Диаграмма последовательностей прецедента AddCourse

На рис. 6.1. показана диаграмма последовательностей, реализующая поведение, описанное прецедентом AddCourse. Согласно спецификации UML2, имена диаграмм взаимодействий могут начинаться с приставки sd, для обозначения того, что данная диаграмма является диаграммой взаимодействий. Довольно странно: sd используется для *всех* типов диаграмм взаимодействий, а не только для диаграмм последовательностей (sd – sequence diagram)!

Время на диаграммах последовательностей развивается сверху вниз, линии жизни выполняются слева направо. Линии жизни размещаются горизонтально, чтобы минимизировать число пересекающихся линий на диаграмме. Их местоположение относительно вертикальной оси отражает момент их создания. Пунктирная линия, находящаяся внизу, показывает существование линии жизни в течение времени.

Вытянутые прямоугольники, расположенные на пунктирной линии под линией жизни, показывают, когда на данной линии жизни находится фокус управления. Эти прямоугольники называются *активациями*, или *фокусом управления*.

0.9. Коммуникационные диаграммы

Коммуникационные диаграммы акцентируют внимание на структурных аспектах взаимодействия, на том, как соединяются линии жизни. В UML2 их семантика довольно слаба по сравнению с диаграммами последовательностей.

Коммуникационные диаграммы акцентируют внимание на структурных аспектах взаимодействия.

Синтаксис коммуникационных диаграмм подобен синтаксису диаграмм последовательностей, за исключением того, что здесь у линий жизни нет «хвостов». Вместо этого они соединены связями, образующими

коммуникационные каналы для передачи сообщений. Порядок сообщений определяется путем иерархической нумерации.

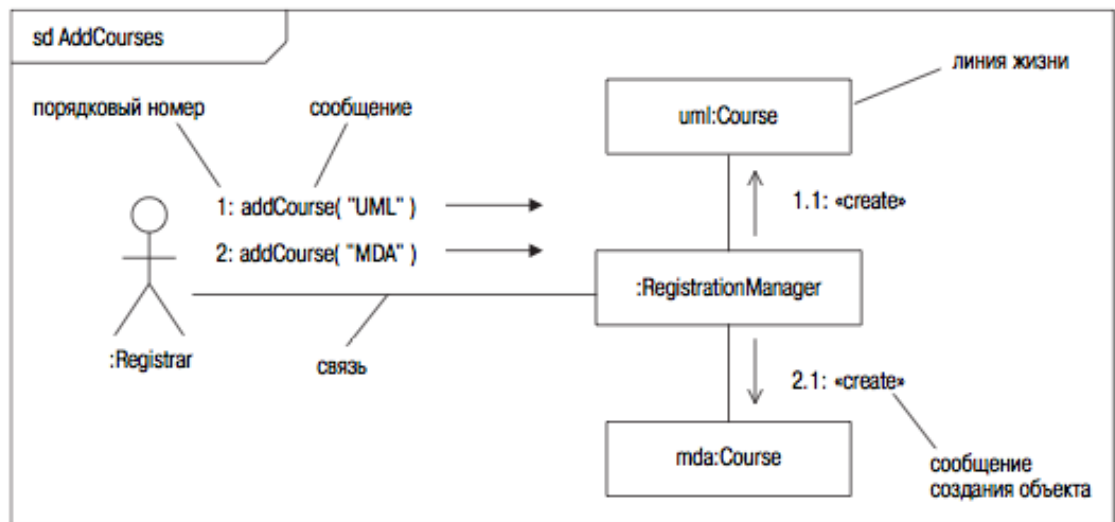


Рис. 6.2. Коммуникационная диаграмма прецедента AddCourses

Главное необходимо понимать, что в результате отправления сообщения вызывается некоторая операция экземпляра и что сложная нумерация сообщений указывает на вложенность вызовов операций (т. е. **вложенный фокус управления**).

0.10.Задание на лабораторную работу.

- Изучить теоретическую часть лабораторной работы.
- Для произвольной программной системы разработать временную диаграмму поведения системы
- Для произвольной программной системы разработать диаграмму развертывания системы
- Оформить отчет по лабораторной работе

0.11. Диаграммы деятельности

В UML2 диаграммы деятельности фактически были лишь особым случаем диаграмм состояний, где у каждого состояния было входное действие, которое определяло некоторый процесс или функцию, имеющие место при входе в состояние. В UML2 диаграммы деятельности имеют совершенно новую семантику, базирующуюся на технологии сетей Петри (Petri Nets). В использовании этой технологии есть два преимущества:

1. Формализм сети Петри обеспечивает большую гибкость при моделировании различных типов потока.
2. В UML2 теперь есть четкое разделение между диаграммами деятельности и диаграммами состояний.

Диаграммы деятельности – это объектно ориентированные блок схемы.

Деятельность может быть добавлена к *любому* элементу модели с целью моделирования его поведения. Элемент обеспечивает контекст для деятельности, и деятельность может использовать возможности своего контекста.

Диаграммы деятельности также могут использоваться для моделирования бизнес процессов и рабочих потоков.

Хорошая диаграмма деятельности сосредоточена на отражении лишь одного определенного аспекта динамического поведения системы. Таким образом, она должна находиться на соответствующем уровне абстракции, чтобы донести эту идею до целевой аудитории, и содержать минимум необходимой информации.

Деятельности – это системы *узлов* (nodes), соединенных *ребрами* (edges). Существует три категории узлов:

- **Узлы действия (action nodes)** – представляют отдельные единицы работы, элементарные *в рамках деятельности*;
- **Узлы управления (control nodes)** – управляют потоком деятельности;
- **Объектные узлы (object nodes)** – представляют объекты, используемые в деятельности.

Ребра представляют потоки деятельности. Существует два типа ребер:

1. **Ребра потоков управления (control flows)** – представляют поток управления деятельностью;
2. **Ребра потоков объектов (object flows)** – представляют поток объектов деятельности.

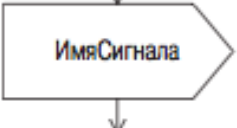
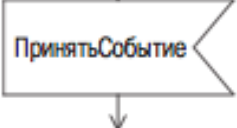
Деятельности обычно начинаются с одного узла управления, начального. Он обозначает начало исполнения при вызове деятельности. Один или более конечных узлов показывают места завершения деятельности.

Обычно диаграммы деятельности используются для моделирования прецедента в виде последовательностей действий.

0.11.1. Узлы действия

Существует четыре типа узлов действия; они перечислены в табл. 7.1.

Таблица 7.1.

Синтаксис	Имя	Семантика
	Узел вызова действия	Иницирует деятельность, поведение или операцию.
	Посылка сигнала	Действие посылки сигнала – посылает сигнал асинхронно (отправитель <i>не</i> ожидает подтверждения получения сигнала). Для создания сигнала может принимать входные параметры.
	Узел действия, принимающий событие	Принимает событие – ожидает события, установленного объектом-владельцем, и выдает событие на выходе. Активируется при получении маркера по входящему ребру. Если <i>нет</i> входящего ребра, запускается при запуске включающей его деятельности и всегда является активированным.
	Узел действия, принимающий событие времени	Принимает событие времени – отвечает на определенное значение времени. Генерирует события времени соответственно своему временному выражению.

0.11.2. Узлы управления

Узлы управления контролируют поток управления деятельности.

Таблица 7.2

Синтаксис	Имя	Семантика	
	Началь- ный узел	Указывает, где начинается поток при вызове деятельности.	
	Конечный узел дея- тельности	Завершает деятельность.	Конечные узлы
	Конечный узел пото- ка	Завершает определенный поток деятельности – другие потоки не затрагиваются.	
«decisionInput» условие принятия решения 	Узел решения	Поток проходит по исходящему ребру, сторожевое условие которого истинно. Может иметь входные данные (необязательно).	
	Узел слияния	Копирует входные маркеры в единственное выходное ребро.	
	Узел ветвления	Разделяет поток на несколько параллельных потоков.	
{описание объединения} 	Узел объ- единения	Синхронизирует несколько параллельных потоков. Может иметь описание объединения (не обязательно) для изменения его семантики.	

0.11.3. Диаграмма деятельности

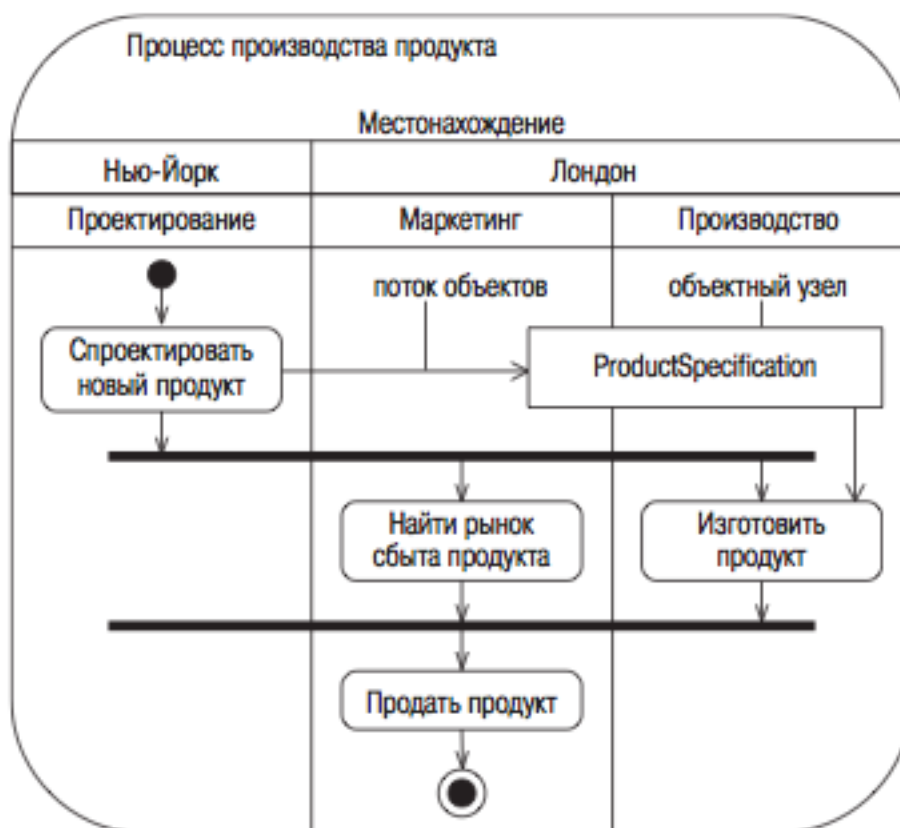


Рис. 7.1. Деятельность

0.11.4. Разъёмы

Основное правило: применения разъёмов необходимо избегать. Однако если диаграмма очень сложна и не поддаётся упрощению, разъёмы можно использовать для разрыва длинных рёбер, которые трудно проследить, и для «распутывания» пересекающихся рёбер. Это может упростить диаграммы деятельности и повысить их удобочитаемость.

Синтаксис разъёма представлен на рис. 7.2. В заданной деятельности каждому исходящему разъёму *должен* соответствовать единственный входящий разъем с такой же меткой. Метки – это идентификаторы разъёма, никакой другой семантики у них нет. Обычно в их качестве выступают буквы алфавита.

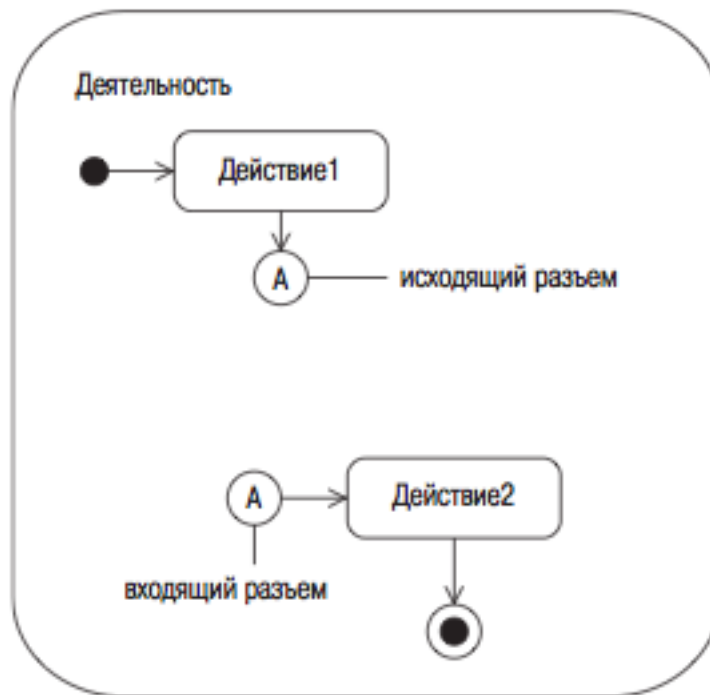


Рис. 7.2. Синтаксис разъёма

0.11.5.Области с прерываемым выполнением действий

Области с прерываемым выполнением действий – это области деятельности, прерываемые при прохождении маркера по прерывающему ребру. Когда область прерывается, все ее потоки немедленно прекращаются. Области с прерываемым выполнением действий полезны для моделирования прерываний и асинхронных событий. Чаще всего они используются при проектировании, но также могут быть полезны при анализе для представления процесса обработки асинхронных бизнес событий.

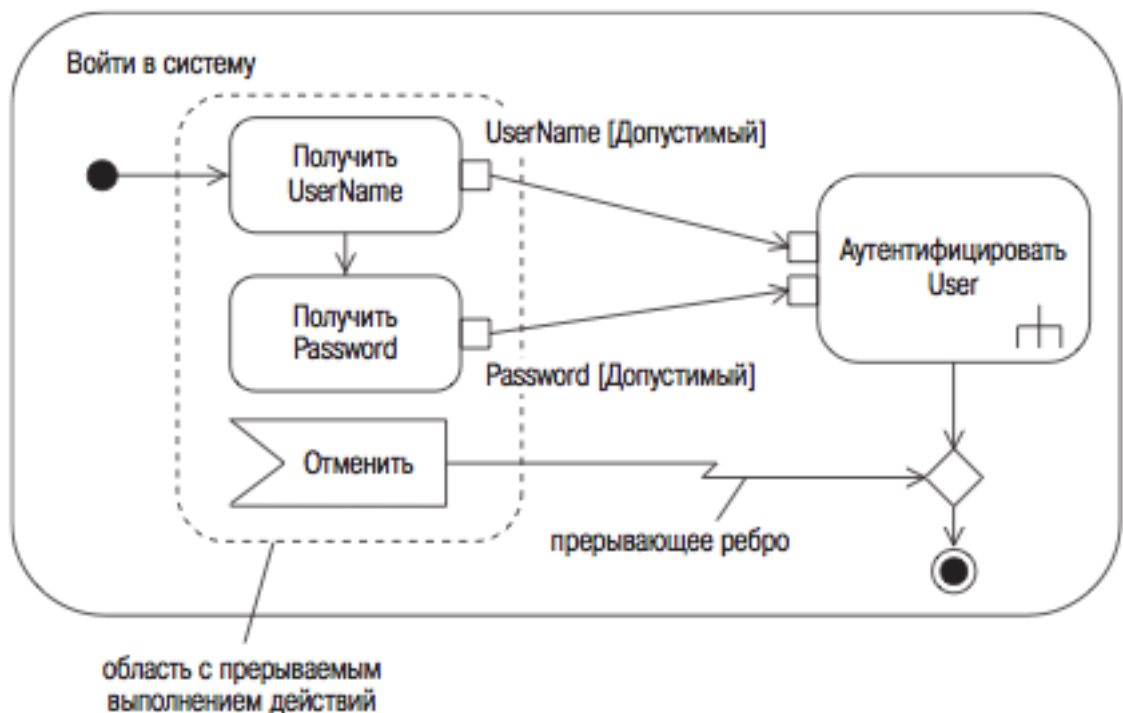


Рис. 6

7.3. Деятельность. Вход в систему имеет область с прерываемым выполнением действий

0.11.6.Обработка исключений

Современные языки программирования часто обрабатывают ошибки посредством механизма, называемого *обработкой исключений* (*exception handling*). В случае выявления ошибки в защищенной части кода создается объект исключения. Поток управления переходит в обработчик исключения, который некоторым образом обрабатывает объект исключения. В этом объекте исключения содержится информация об ошибке, которая может использоваться обработчиком исключения. Обработчик исключения может прервать приложение или попытаться восстановить нормальное состояние. Часто информация объекта исключения сохраняется в журнале регистрации ошибок.

Обработку исключений на диаграммах деятельности можно моделировать с помощью контактов исключений, защищенных узлов и обработчиков исключений.

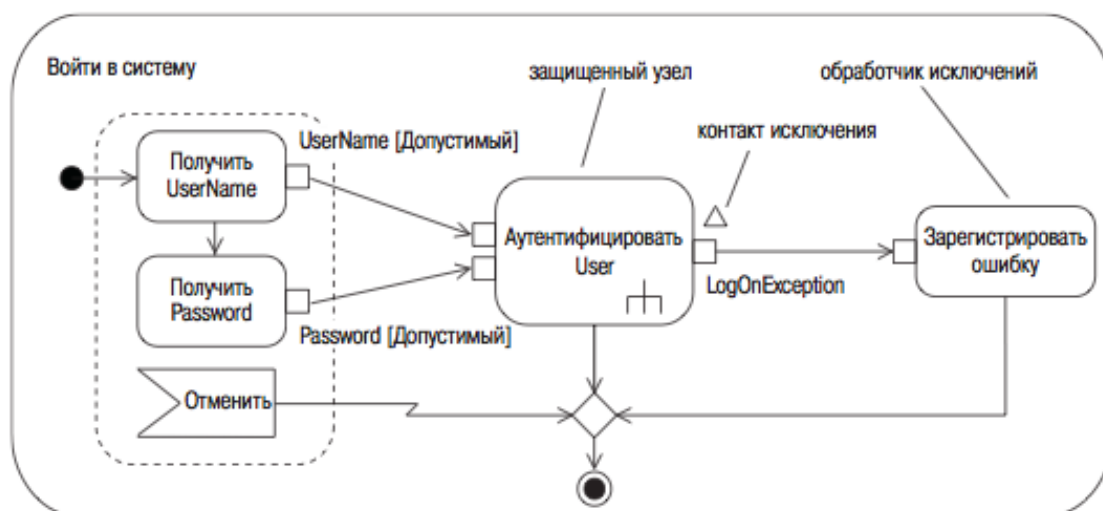


Рис. 7.4. Деятельность. Вход в систему с обработчиком исключений

Для того чтобы показать, что выходной контакт представляет собой объект исключения, его помечают небольшим равнобедренным треугольником (рис. 7.4). Узел Зарегистрировать ошибку выступает в роли обработчика исключений, генерируемых действием Аутентифицировать User. Если с узлом ассоциирован обработчик исключений, узел называют защищенным.

0.11.7.Узлы расширения

Узел расширения – это объектный узел, который представляет коллекцию объектов, входящую в или выходящую из области расширения. Область расширения выполняется по одному разу для каждого входного

элемента. На рис. 7.5 показан пример области расширения. Она представлена в виде отрисованного пунктиром прямоугольника со скругленными углами и входящим и исходящим узлами расширения. Узел расширения выглядит как контакт, но с тремя квадратными ячейками. Это указывает на поступление коллекции, а не одного объекта.

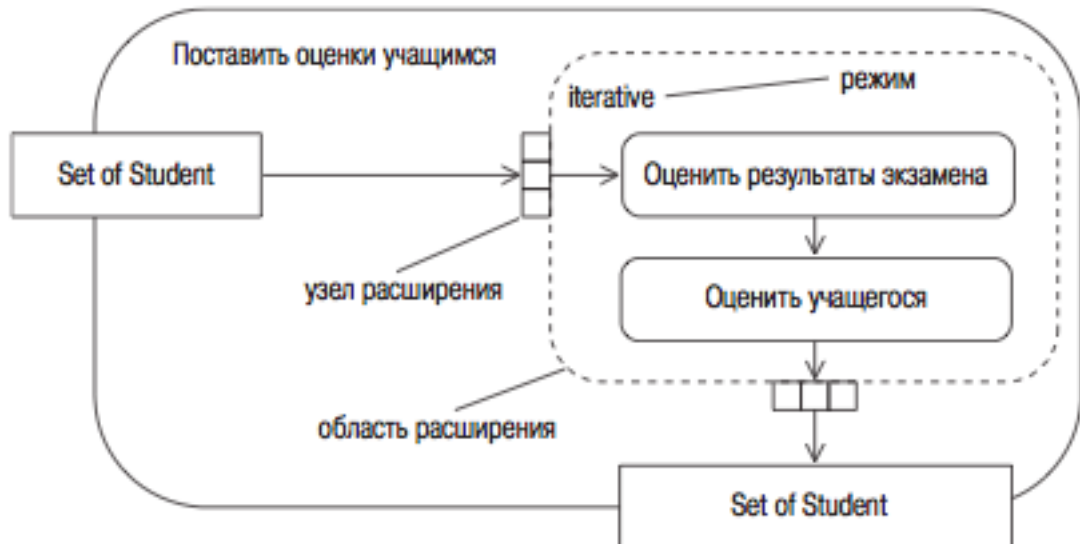


Рис. 7.5. Пример области расширения

На узлы расширения накладываются два ограничения.

- Тип выходной коллекции *должен* соответствовать типу входной коллекции.
- Типы объектов входной и выходной коллекций *должны* быть одинаковыми. ^[SEP] Эти ограничения означают, что области расширения не могут использоваться для преобразования входных объектов одного типа в выходные объекты другого типа. Количество выходных коллекций может не совпадать с количеством входных, поэтому области расширения могут использоваться для объединения или разделения коллекций.

0.11.8.Отправка сигналов и приём событий

Сигнал представляет асинхронно передаваемую между объектами информацию. Сигнал моделируется как класс, отмеченный стереотипом «signal» (сигнал). Передаваемая информация хранится в атрибутах сигнала. При анализе сигналы могут использоваться для отображения отправки и получения асинхронных бизнес событий (таких как OrderReceived (заказ получен)), а при проектировании они могут иллюстрировать асинхронный обмен информацией между разными системами, подсистемами или частями оборудования.

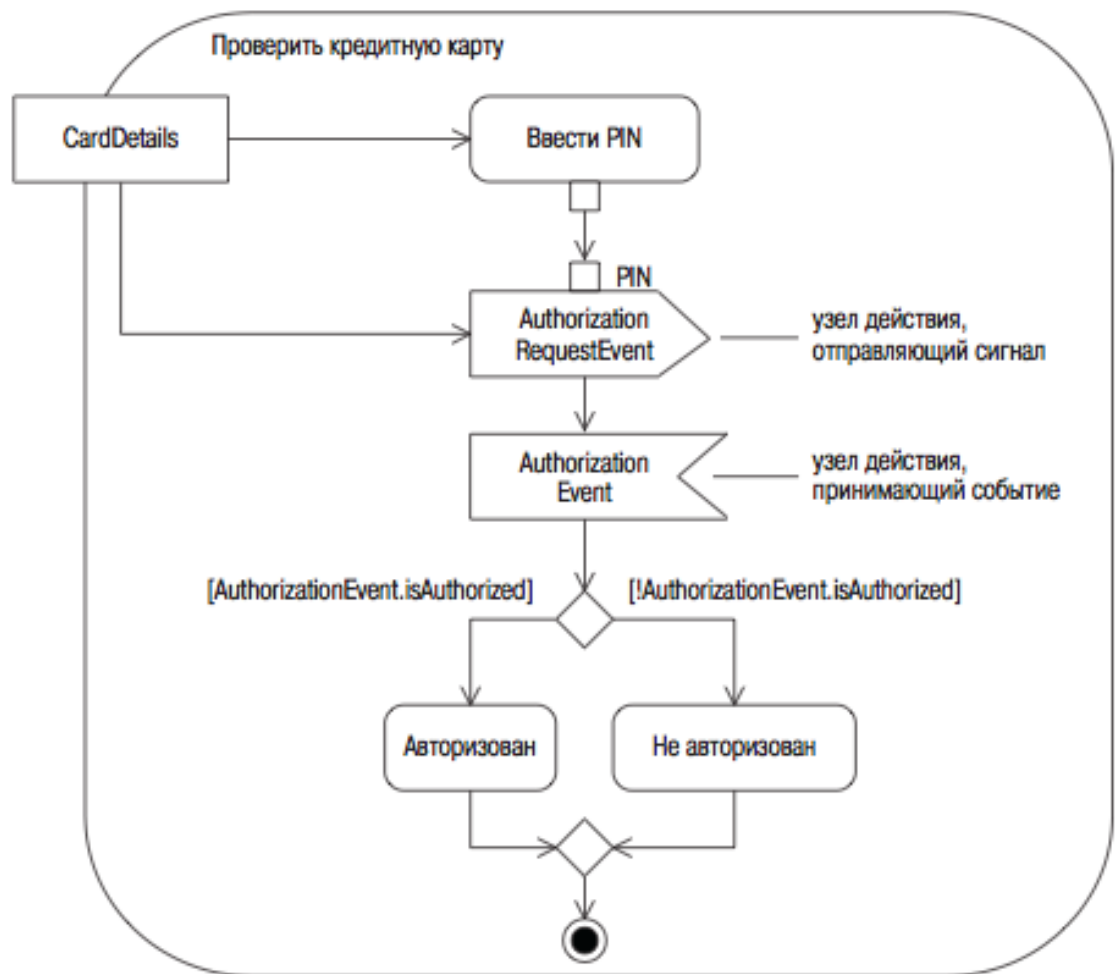


Рис. 7.6. Деятельность Проверить кредитную карту

Сигнал можно послать с помощью узла действия, отвечающего за отправку сигнала. Он посылает сигнал асинхронно – деятельность, отправляющая сигнал, *не* ожидает подтверждения получения сигнала.

0.11.9.Потоковая передача

Обычно действия принимают маркеры со своих входных ребер только тогда, когда начинают выполнение, и предлагают их на своих выходных ребрах по завершении выполнения. Однако иногда требуется, чтобы действие выполнялось *непрерывно*, периодически принимая и предлагая маркеры. Такое поведение называют потоковой передачей. В UML2 оно может быть проиллюстрировано любым из четырех способов, как показано на рис. 7.7.

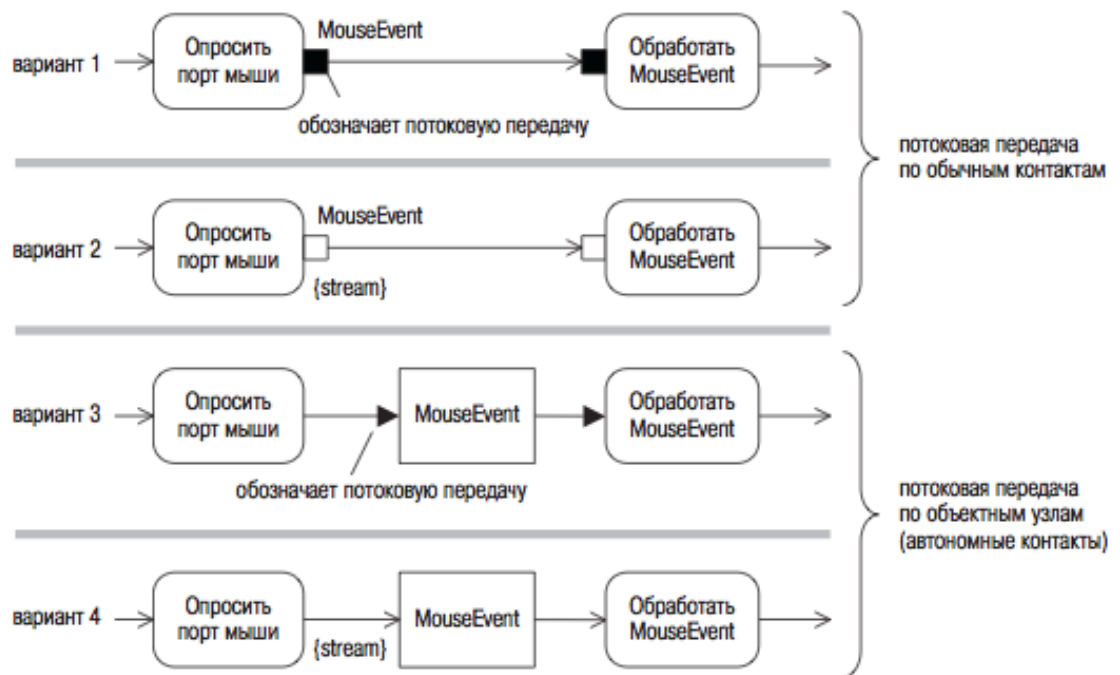


Рис. 7.7. Четыре способа отображения потоковой передачи

Любая ситуация, в которой необходимо *непрерывно* получать и обрабатывать информацию, является кандидатом на использование потоковой передачи.

0.12. Диаграммы обзора взаимодействий

Диаграммы обзора взаимодействий – особая форма диаграммы деятельности. Они показывают взаимодействия и включения взаимодействий (interaction occurrences) и используются для моделирования высокоуровневого потока управления между взаимодействиями.

Одно особенно мощное применение диаграмм обзора взаимодействий – иллюстрация потока управления между прецедентами. Если каждый прецедент представить как взаимодействие, синтаксис диаграммы деятельности можно использовать для отображения движения потока управления между ними.

Обратите внимание, что линии жизни, принимающие участие во взаимодействии, могут быть перечислены после ключевого слова *lifelines* (линии жизни) в заголовке диаграммы. Такое документирование может быть полезным, поскольку часто линии жизни скрыты внутри включений взаимодействий.

Синтаксис диаграмм обзора взаимодействий аналогичен синтаксису диаграмм деятельности, за исключением того, что здесь отображаются встроенные взаимодействия и включения взаимодействий, а не деятельности и объектные узлы.

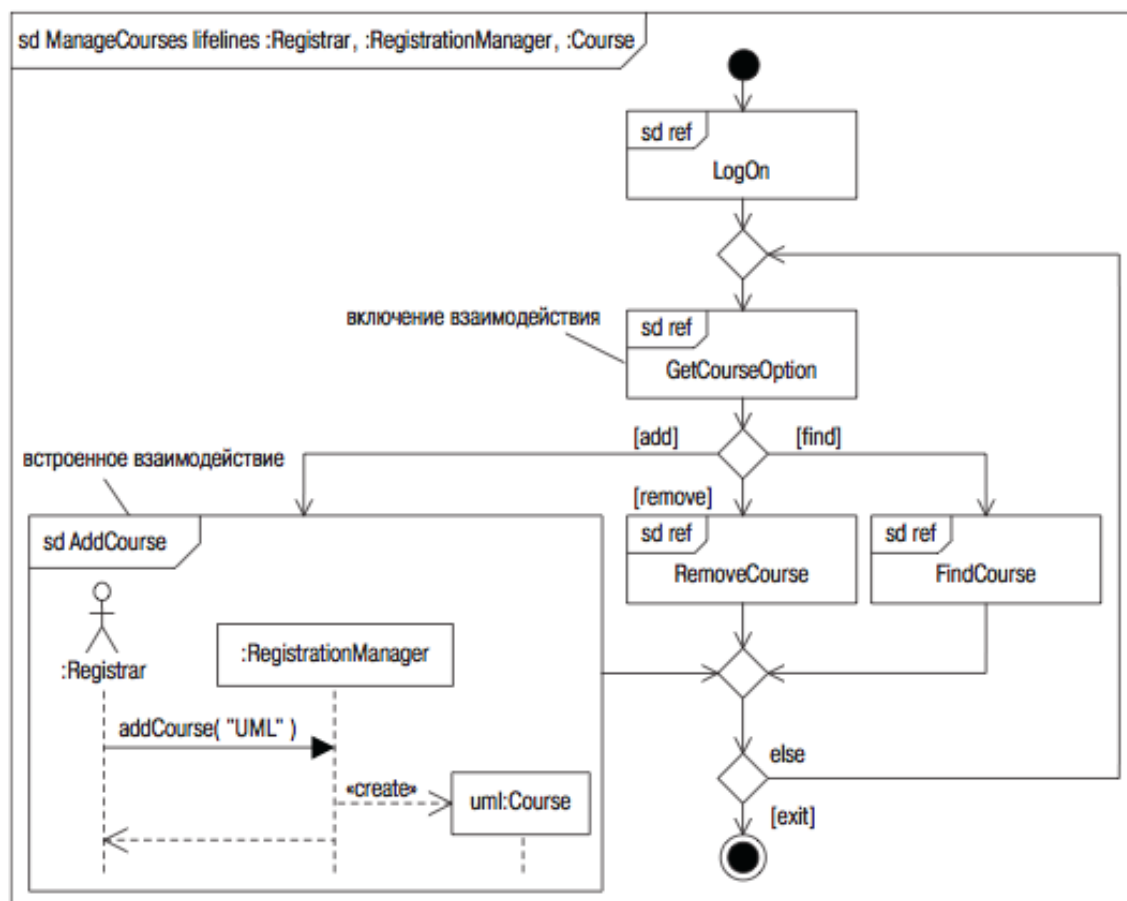


Рис. 7.8. Диаграмма обзора взаимодействий

Диаграммы обзора взаимодействий дают визуальное представление ветвления, параллелизма и итерации, что намного понятней и проще для восприятия. В то же время они переносят ваше внимание с элементов, например линий жизни, непосредственно на поток управления. Поэтому эти диаграммы должны использоваться, когда необходимо акцентировать внимание на движении потока управления через множество взаимодействий.

В реализации прецедента взаимодействия описывают поведение, определенное в прецеденте. Таким образом, диаграммы обзора взаимодействий могут использоваться для иллюстрации бизнес процессов, охватывающих прецеденты.

0.13.Задание на лабораторную работу.

- Изучить теоретическую часть лабораторной работы.
- Для произвольной программной системы описать динамику поведения программной системы.
- Обормить структуру программ в виде динамических диаграмм.
- Оформить отчет по лабораторной работе

0.14. Конечные автоматы

Конечный автомат моделирует динамическое поведение реактивного объекта.

- Спецификация UML2 определяет два типа конечных автоматов, имеющих общий синтаксис:

поведенческие автоматы;

протокольные автоматы.

Поведенческие автоматы с помощью состояний, переходов и событий определяют *поведение* контекстного классификатора. Они могут использоваться, только если у контекстного классификатора есть некоторое поведение, которое можно смоделировать. У некоторых классификаторов, например интерфейсов и портов, такого поведения нет; они просто описывают протокол использования. Состояния поведенческих автоматов могут определять одно или более действий, выполняемых при входе в состояние, нахождении в нем или выходе из него.

Протокольные автоматы используют состояния, переходы и события для определения *протокола* контекстного классификатора. Протокол включает:

- условия, при которых могут вызываться операции классификатора и его экземпляров;
- результаты вызовов операций;
- порядок вызовов операций. ^[1]_{SEP}

Протокольные автоматы ничего не сообщают о реализации этого поведения. Они только определяют, как оно представляется внешней сущности. Протокольные автоматы могут использоваться при описании протокола для *всех* классификаторов, включая те, которые не имеют реализации. Состояния протокольных автоматов *не могут* определять действия; это дело поведенческих автоматов.

0.15. Диаграммы состояний

Для иллюстрации диаграмм состояний (state machine diagrams) давайте рассмотрим простой пример. Один из самых наглядных объектов реального мира, который постоянно переходит из состояния в состояние, – электрическая лампочка. На рис. 8.1 показана передача событий от переключателя к лампочке. Могут быть посланы два события: turnOn (включить) (это событие моделирует подключение лампы в электрическую сеть) и turnOff (выключить) (выключает ток).

Диаграмма состояний содержит только один конечный автомат для единственного реактивного объекта. В данном случае реактивный объект – это система, состоящая из лампочки, переключателя и электропитания. Диаграмма состояний может изображаться в явно обозначенной рамке, как показано на рис. 8.1, или существовать в неявных рамках, предоставляемых средством моделирования.

У каждого конечного автомата должно быть начальное состояние (закрашенный кружок), обозначающее первое состояние последовательности.

Если смена состояний не бесконечна, должно присутствовать и конечное состояние (бычий глаз), которое завершает последовательность переходов. Обычно переход от начального псевдосостояния к первому «настоящему» состоянию происходит автоматически. Начальное псевдосостояние используется просто как удобный маркер для обозначения начала ряда переходов состояний.

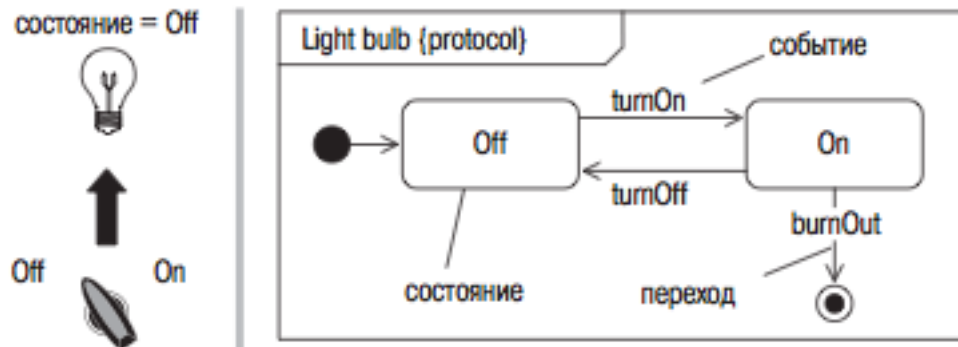


Рис. 8.1. Диаграмма состояний электрической лампочки

0.15.1. Синтаксис состояния

UML2 синтаксис состояния представлен на рис. 8.2.

Каждое состояние поведенческого автомата может содержать нуль или более действий и деятельности. У состояний протокольных автоматов *нет* действий или деятельности.

Действия считаются мгновенными и непрерываемыми, тогда как деятельности занимают конечное время и могут быть прерваны. Каждое действие в состоянии ассоциируется с внутренним переходом, инициируемым событием. В состоянии может быть любое число действий и внутренних переходов.

Внутренний переход позволяет зафиксировать тот факт, что произошло что-то, заслуживающее отражения в модели, но не обуславливающее (или не настолько важное, чтобы моделировать это как) переход в новое состояние. Например, на рис. 8.2 нажатие одной из клавиш клавиатуры, конечно, является заслуживающим внимания событием, но оно не приводит к переходу из состояния EnteringPassword (ввод пароля). Мы моделируем это как внутреннее событие keypress (нажатие клавиши), которое обуславливает внутренний переход, инициирующий действие отобразить “*“.

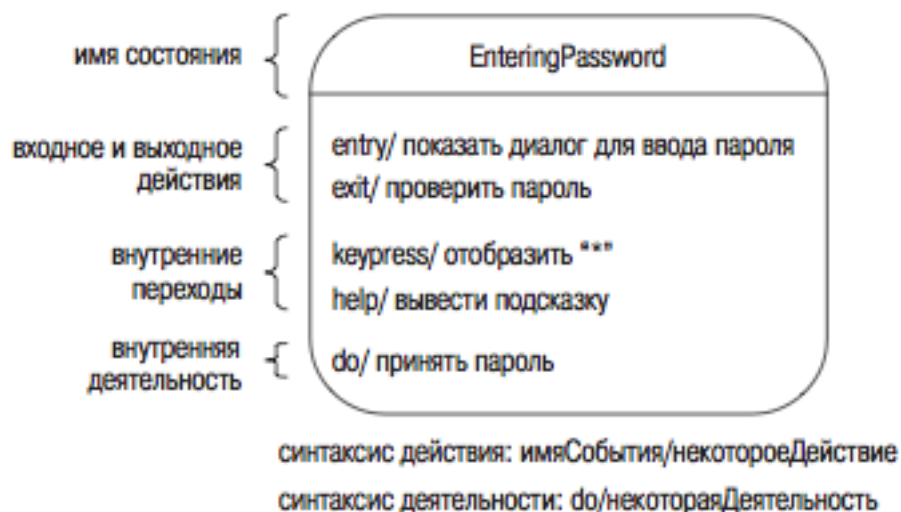


Рис. 8.2. Синтаксис состояния

Два специальных действия – вход и выход – ассоциированы со специальными событиями entry и exit. У этих двух событий особая семантика. Событие entry происходит мгновенно и автоматически при входе в состояние. Это первое, что происходит, когда осуществляется вход в состояние. Это событие обуславливает выполнение ассоциированного с ним действия на входе. Событие exit – самое последнее, что происходит мгновенно и автоматически при выходе из состояния. Обуславливает выполнение ассоциированного действия на выходе.

0.15.2. Переходы

UML2 синтаксис переходов для поведенческих автоматов представлен на рис. 8.3.

Переходы показывают движение между состояниями.

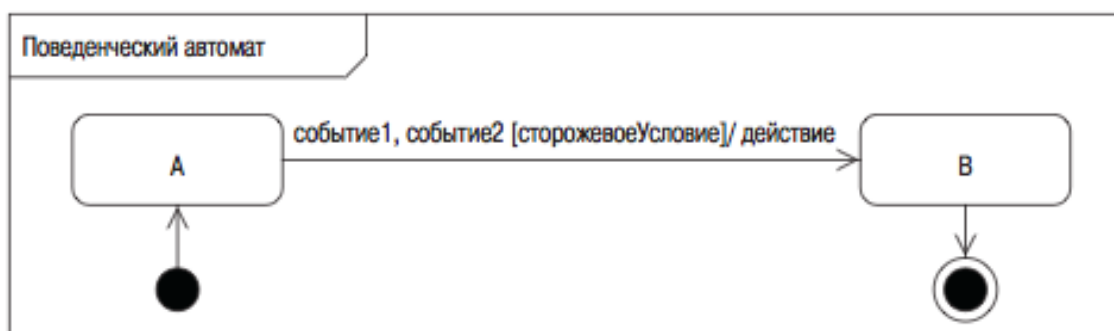


Рис. 8.3. Синтаксис переходов для поведенческих автоматов

Синтаксис переходов в поведенческом автомате прост и может использоваться для внешних переходов (изображаются стрелками) или внутренних переходов (изображаются вложенными в состояние). Каждый переход имеет три необязательных элемента.

1. Нуль или более событий – определяют внешнее или внутреннее происшествие, которое может инициировать переход.
2. Нуль или одно сторожевое условие – логическое выражение, которое должно быть выполнено (иметь значение true), прежде чем может произойти переход. Условие указывают после событий.
3. Нуль или более действий – часть работы, ассоциированная с переходом и выполняемая при срабатывании перехода.

0.15.3.События

События инициируют переходы. UML2 определяет событие как «описание заслуживающего внимания происшествия, занимающего определенное положение во времени и пространстве». События инициируют переходы в автоматах. События могут быть указаны вне состояний на переходах, или внутри состояний.

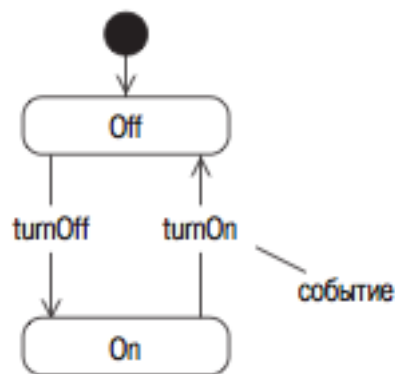


Рис. 8.4. События указаны вне состояний на переходах

Существует четыре семантически различных типа событий:

- событие вызова; [SEP]
- сигнал;
- событие изменения;
- событие времени.

0.15.4.Составные состояния

Составное состояние – это состояние, содержащее вложенные состояния.

Эти вложенные состояния объединяются в один или более конечных автоматов, которые называют *подавтоматами* (submachines). Для каждого подавтомата в пиктограмме композиции отведена собственная *область*. Области – это просто участки пиктограммы состояния, разделенные пунктирными линиями.



Рис. 8.5. Каждому подавтомату отведена своя область

Вложенные состояния также могут быть составными состояниями. Однако, как правило, необходимо по возможности стремиться к тому, чтобы вложенность составных состояний не превышала двух или трех уровней. При большей глубине вложенности автомат сложно воспринимать на диаграмме и понимать.

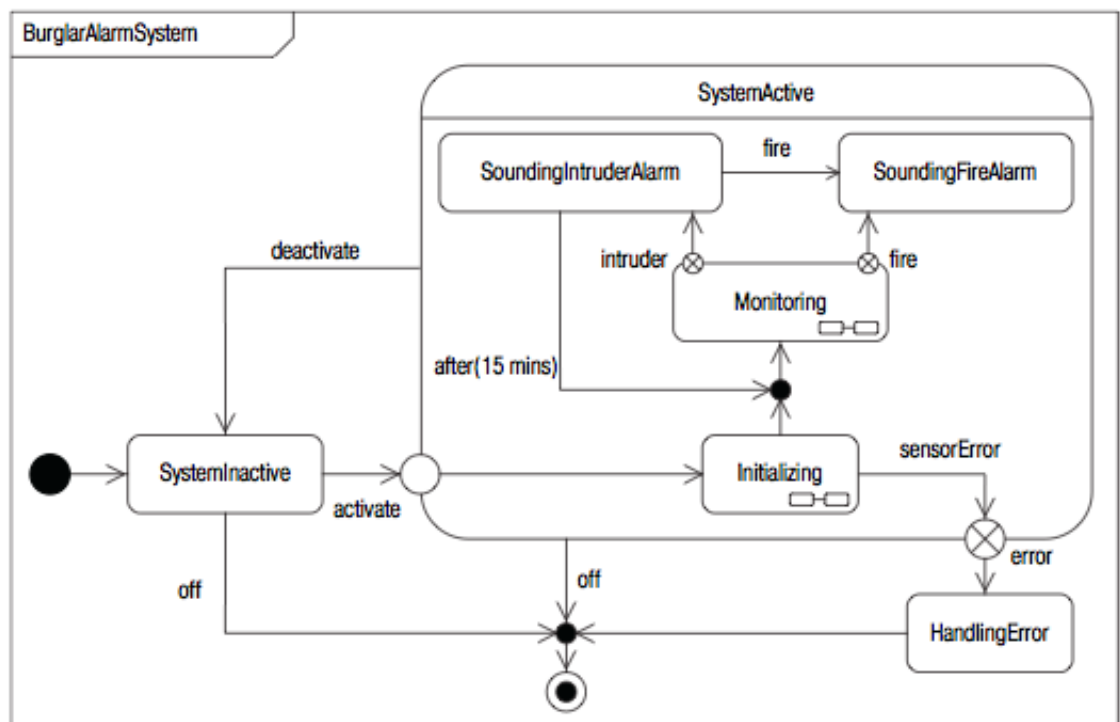


Рис. 8.6. Автомат системы сигнализации

При входе в это состояние происходит ветвление и начинается параллельное выполнение двух подавтоматов. В верхнем подавтомате состояние InitializingFireSensors выполняет процесс инициализации пожарных датчиков. В нижнем подавтомате состояние InitializingSecuritySensors делает то же самое для датчиков безопасности.

В нормальных условиях по завершении *обоих* подавтоматов происходит автоматический выход из суперсостояния Initializing. Это объединение:

подавтоматы синхронизируются таким образом, что дальнейшая работа невозможна, пока не будут инициализированы *и* пожарные датчики, *и* датчики безопасности.

0.16. Временные диаграммы

Одним из слабых мест UML2 было моделирование систем реального времени. Это такие системы, в которых временные соотношения критически важны и события должны следовать друг за другом в рамках определенного временного окна. Мы говорим «временное окно», а не «время», потому что абсолютное время для нас как разработчиков *неприемлемо*. Когда в модели задается время, на самом деле задается время плюсминус некоторая погрешность, определяемая внешними факторами, такими как точность системных часов. Обычно это не является проблемой, за исключением систем с очень точными временными ограничениями.

Временные диаграммы очень просты. Время откладывается на горизонтальной оси слева направо. Линии жизни и их состояния (или определенные условия, накладываемые на линии жизни) располагаются вертикально. Переходы между состояниями линий жизни и условиями представляются в виде графика.

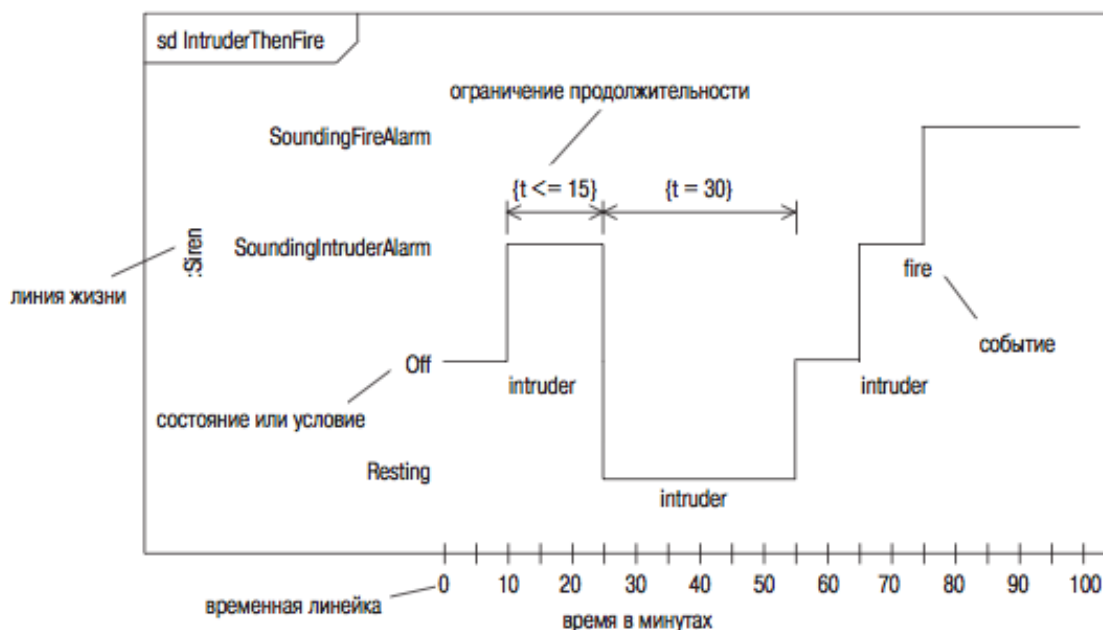


Рис. 8.7. Временная диаграмма

Временные диаграммы также можно представить в более компактной форме, когда состояния располагаются горизонтально.

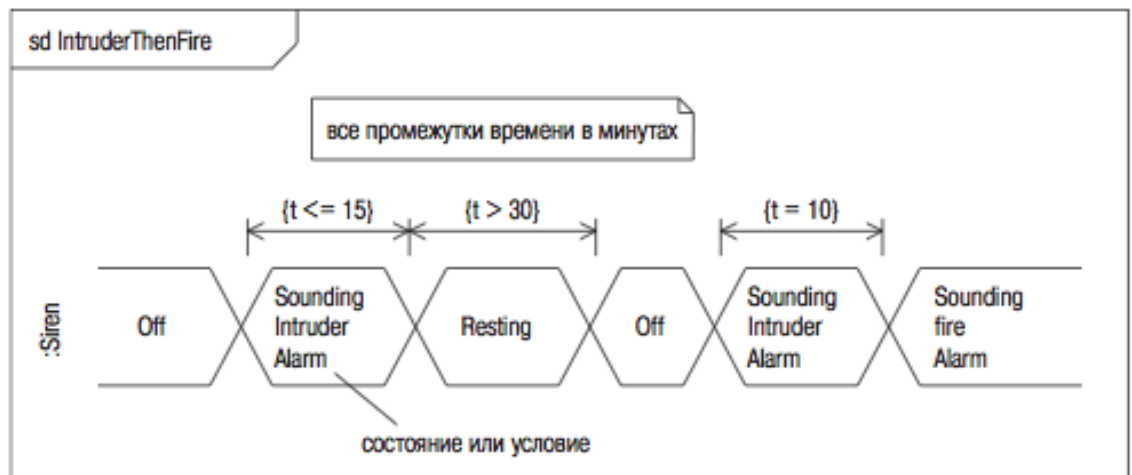


Рис. 7.8. Компактная форма временной диаграммы

0.17.Задание на лабораторную работу.

- Изучить теоретическую часть лабораторной работы.
- Для произвольной программной системы описать динамику поведения программной системы с помощью автоматной схемы.
- Оформить структуру программ в виде диаграммы состояний.
- Оформить отчёт по лабораторной работе